

Simple Shell Documentation

Table of Contents

1. Introduction
2. Features
3. Getting Started
4. Organisation
5. Shell Commands
6. Background Processes and Taking input from an external file as commands (BONUS)
7. Command History
8. Built-in Commands
9. Error Handling
10. Contributors
11. Conclusion

1. Introduction

"Aggaris", is a Unix-like shell . It offers a range of features for executing both built-in and external commands, managing background processes, and maintaining a history of executed commands.

2. Features

It offers the following key features:

- **Command Execution:** Execute various shell commands, both built-in (e.g., `ls`, `echo`, `history`) and external commands.
- **Background Processes:** Run commands in the background using the `'&'` symbol, allowing for concurrent execution. (**Bonus**)
- **Command History:** Maintain a history of executed commands, viewable with additional execution details.
- **Built-in Commands:** Implementations for common Unix-like commands like `ls`, `echo`, `grep`, `wc`, and more.
- **Error Handling:** Robust error handling ensures the shell operates reliably, providing informative error messages when needed.

3. Getting Started

To use It in your system, follow these steps:

1. Compile the code using a C compiler

```
gcc simple-shell.c
```

2. Run the shell using the compiled binary

`./a.out`

3. You'll be greeted with a command prompt, ready to accept your commands.

4. Organisation

The structure of the directory is given below:

```
Documentation.txt
fib.c
file.txt
helloworld.c
simple-shell.c
```

5. Shell Commands

It supports both built-in and external commands. You can execute common shell commands like `ls`, `echo`, `grep`, and others.

Example usage:

- To list files in the current directory, type `ls`.
- To print a message, type `echo Hello World!`.

6. Background Processes and Taking input from an external file as commands(BONUS)

You can run commands in the background by appending the `&` symbol at the end of the command. This allows for concurrent execution of multiple tasks.

Example usage:

```
sleep 10 &
```

It will run the `sleep 10` command in the background for 10 seconds and after 10 seconds you will see a process ID (PID) printed on the terminal.

7. Command History

It maintains a history of previously executed commands. To view your command history, simply type `history` at the shell prompt.

8. Built-in Commands

The shell includes several built-in commands for enhanced functionality:

- `ls`: List files and directories.
- `echo`: Display a message.
- `exit`: Exit the shell.

To execute a built-in command, simply type the command name followed by any required arguments.

Example usage:

- Type `ls -l` to list files in long format.
- Type `echo Hello Shell!` to display a custom message.

9. Error Handling

It incorporates error handling to provide informative error messages in case of issues. Error messages are displayed for various situations, such as failed command execution, input validation errors, and more.

10. Contributors

- Gagan Raj Singh: Contributions include core shell functionality, process management, and documentation.
- Garv Sachdeva: Contributions include command history, user interface, and built-in commands implementation.

11. Conclusion

It is a Simple Unix shell that offers a range of features for efficient command execution and management. It provides a user-friendly command-line interface and supports both built-in and external commands.